IN @arasu_oracle_dba

ORACLE ON LINUX · THE MEMORY LAYER

How Oracle MEMORY really works on Linux

SGA, PGA, page tables, HugePages, swap, AMM vs ASMM — and one 500 GB server sized two ways: with HugePages, and without.

WHAT'S INSIDE

1. Linux memory in one page
2. Where 500 GB goes
3. SGA pools & granules
4. PGA target vs limit
5. The page-table trap
6. HugePages on RHEL
7. use_large_pages = ONLY
8. Swap used vs paging
9. AMM vs ASMM vs Manual
10. 500 GB worked example
11. Same box, no HugePages
12. A free sizing calculator

How to read this:

DOC = Oracle Installation Guide, DBA Reference for Linux or Database Reference. PRACTICE = practitioner rule of thumb that I use — adjust it to your workload.

SWIPE TO NEXT →

01 · LINUX FIRST

Oracle memory is just Linux memory

Every Oracle memory problem is first a Linux memory problem: the SGA is shared memory, the PGA is anonymous memory.

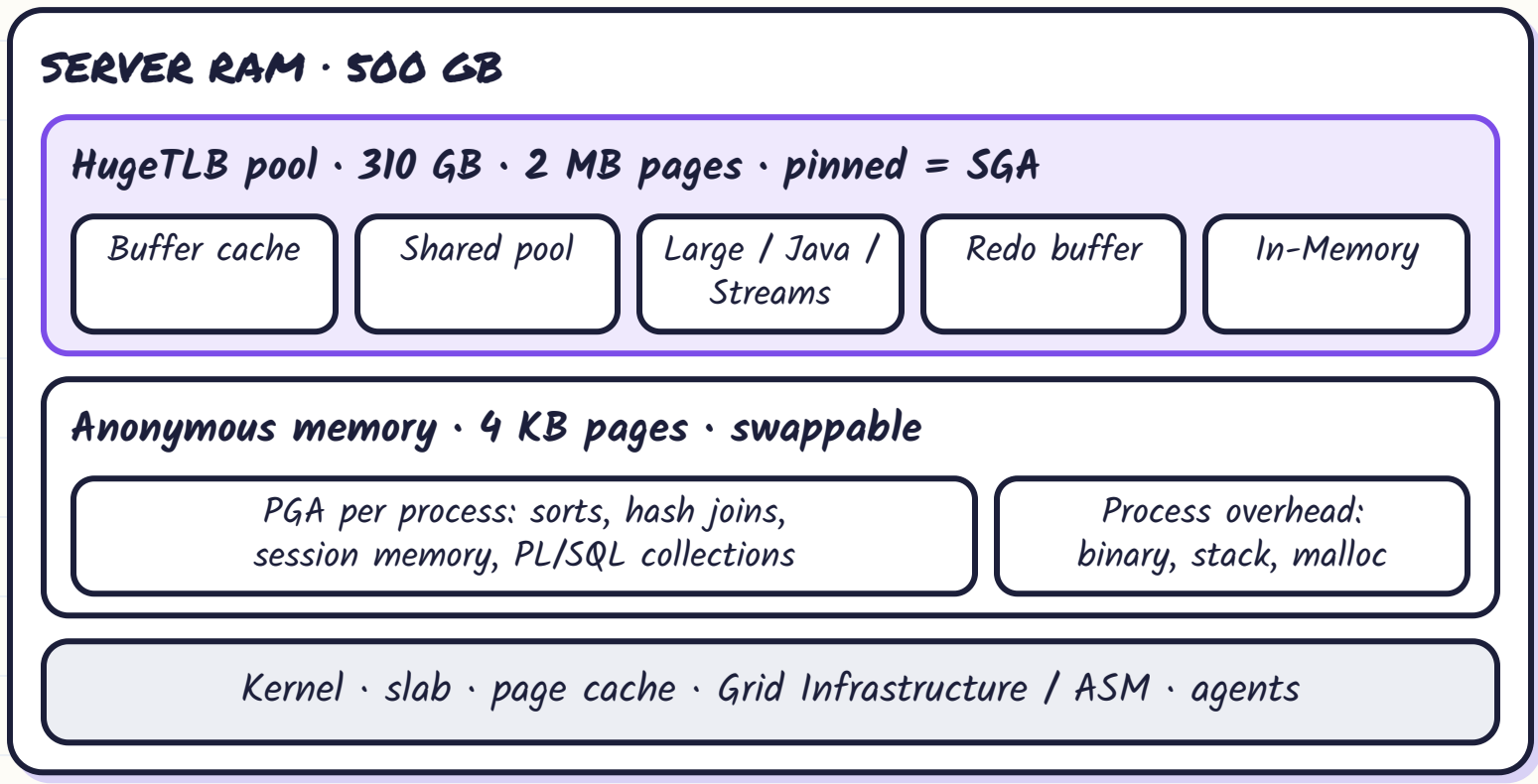
Term	What it is	Why a DBA cares
Page	Smallest unit the kernel maps: **4 KB**	Each mapped page costs one 8-byte page-table entry **per process**
TLB	CPU cache of recent mappings, a few thousand entries	4 KB pages cover ~6 MB; a 310 GB SGA misses constantly
Shared memory	One SysV shm segment mapped by many processes	This is the **SGA** (`ipcs -m`)
Anonymous memory	Private heap and stack of a process	This is the **PGA** and process overhead · swappable
Page cache	Kernel cache of file blocks	Near zero on ASM; large on filesystem DBs without `setall`
HugeTLB pool	Pre-reserved **2 MB** pages, pinned, never swapped	Where the SGA should live; shows as *used* in `free` from boot
Swap	Disk overflow for anonymous pages	A safety margin against the OOM killer, not a working area
THP	Kernel auto-promotes anonymous memory to 2 MB	**Disable it** — THP is not HugePages

Two different things: `transparent_hugepage=never` on the kernel line DOC and an explicit `vm.nr_hugepages` pool. You turn THP **off** and HugePages **on**.

SWIPE TO NEXT →

02 · THE MAP

WHERE 500 GB GOES



ANONYMOUS PAGES ONLY → **Swap · 16 GB on disk**

HugePages are pinned: they never page out

The one rule: *SGA + `PGA_AGGREGATE_LIMIT` + overhead must fit in RAM. The SGA cannot swap; the PGA is the only Oracle memory that can — so the limit is what keeps the host out of swap.*

```
# see this map on a live host
free -g
grep -E 'Shmem|HugePages_(Total|Free)|PageTables|Swap' /proc/meminfo
ipcs -m -u          # the SGA segments
```

SWIPE TO NEXT →

03 · SGA

One shared segment, many pools

Pool	Parameter	Holds	ASMM moves it?
Buffer cache	`DB_CACHE_SIZE`	Data blocks	Yes
Shared pool	`SHARED_POOL_SIZE`	SQL, plans, dictionary, result cache	Yes
Large pool	`LARGE_POOL_SIZE`	RMAN, PX messages, shared server	Yes
Java / Streams	`JAVA_POOL_SIZE` `STREAMS_POOL_SIZE`	JVM · GoldenGate, AQ, Data Pump	Yes
Redo log buffer	`LOG_BUFFER`	Redo before LGWR writes it	No, static
In-Memory	`INMEMORY_SIZE`	Columnar copies, carved from SGA_TARGET	No, static
KEEP / RECYCLE / nK	`DB_KEEP_CACHE_SIZE …`	Pinned, throwaway, other block sizes	No, static

Two facts that decide your hugepages count DOC

- `SGA_MAX_SIZE` creates the shm segment **at startup**, even if `SGA_TARGET` is lower. Size the pool from `SGA_MAX_SIZE`, never from `SGA_TARGET`.
- The SGA moves in **granules**: 4 MB for a tiny SGA, **512 MB above 128 GB** (MOS 947152.1). On a 310 GB SGA the smallest resize is half a GB.

```
SELECT component, current_size/1073741824 gb, last_oper_type
  FROM v$sga_dynamic_components;   -- what ASMM has been doing
```

SWIPE TO NEXT →

04 · PGA

PRIVATE, PER PROCESS, 4 KB PAGES

WHAT LIVES IN IT

- Work areas: sort, hash join, bitmap merge
- Private SQL area, cursor state
- Session memory (UGA)
- PL/SQL collections

NOT IN SGA OR PGA

Process overhead: binary text, stack, malloc heap ≈ **5–20 MB per process**. 2,000 processes ≈ **20 GB** that no Oracle parameter counts. PRACTICE

TARGET VS LIMIT DOC

```
PGA_AGGREGATE_TARGET → target, tunable work areas only
PGA_AGGREGATE_LIMIT → hard ceiling, ORA-04036 past it
limit default = MAX(2 GB, 2 × target, 3 MB × PROCESSES)
```

- Real PGA can exceed the target: PL/SQL collections and cursors are **untunable**
- Every process carries ~3–5 MB of baseline whether busy or idle PRACTICE
- Set the limit **explicitly** so SGA + limit + overhead ≤ RAM, instead of trusting 2×

Size against evidence: `v$pgastat` 'maximum PGA allocated' since startup, and `v$process` sorted by `pga_alloc_mem`.

SWIPE TO NEXT →

05 · WHY HUGEPAGES

THE PAGE-TABLE TRAP

*The SGA is **shared**: every server process maps it. With 4 KB pages each process builds its **own** page tables for the part of the SGA it touches.*

310 GB SGA	4 KB pages	2 MB HugePages
Pages in the SGA	*81,264,640*	***158,720***
Page tables per fully-attached process	***≈ 620 MB***	***≈ 1.2 MB***
1,500 processes, all touched	***≈ 930 GB** — nearly twice the box*	*a few GB, and hugetlbfs shares them*
TLB coverage (~1,536 entries)	*≈ 6 MB of SGA*	*≈ 3 GB of SGA*

```
page tables per process = SGA / 4096 × 8 B ≈ SGA / 512
```

WHAT IT LOOKS LIKE IN PRODUCTION PRACTICE

1. *Startup is fine. `free -g` looks fine.*
2. *`PageTables` in `/proc/meminfo` climbs for hours as sessions touch the cache*
3. *It eats the overhead budget, then the kernel pages **PGA** out; sessions stall on page faults*

The classic ticket: "the server has 500 GB and it is swapping." Nothing leaks. The SGA is on the wrong page size.

SWIPE TO NEXT →

06 · HUGEPAGES ON RHEL

SIX STEPS, ONE REBOOT

1. Pages = **ceil(Σ SGA_MAX_SIZE / 2 MB) + ~5 per instance**. Oracle's `hugepages_settings.sh` (MOS 401749.1) computes it from running instances.
2. Kernel parameters, then `sysctl -p`

```
# /etc/sysctl.d/99-oracle.conf   (310 GB SGA)
vm.nr_hugepages      = 158770       # 310*512 + 50
vm.hugetlb_shm_group = 54321        # gid of oinstall
kernel.shmmax        = 332859965440 # >= SGA_MAX_SIZE bytes
kernel.shmall        = 81264640     # >= sum SGA / 4096
```

3. `memlock` for **oracle and grid**: at least 90% of RAM when HugePages are used, never below the pool DOC

```
# /etc/security/limits.conf  (KB)
oracle  soft  memlock  472000000
oracle  hard  memlock  472000000
grid    soft  memlock  472000000
grid    hard  memlock  472000000
```

4. `transparent_hugepage=never` on the kernel line DOC
5. **Reboot**: a large pool rarely allocates cleanly on a running system
6. Verify: `grep -i huge /proc/meminfo` and `ulimit -l` as oracle

Forgot grid? The ASM instance cannot lock its pages and the whole stack falls back to 4 KB.

SWIPE TO NEXT →

07 · THE ORACLE SIDE

USE_LARGE_PAGES = ONLY

Value	Behaviour	Use it
`FALSE`	*Never uses HugePages*	***Never***
`TRUE` *(default)*	*Uses what is available, puts the rest in 4 KB pages* ***silently***	***Dangerous***
`ONLY`	*All or nothing: startup* ***fails*** *if the pool is short*	***Production***
`AUTO_ONLY` *(19c+)*	*Instance grows the pool itself via* `oradism`*; fails if it cannot*	*Consolidation nodes, after testing on your RU*

```
ALTER SYSTEM SET use_large_pages = 'ONLY' SCOPE=SPFILE;
ALTER SYSTEM SET sga_max_size = 310G SCOPE=SPFILE;
ALTER SYSTEM SET sga_target   = 310G SCOPE=SPFILE;
ALTER SYSTEM RESET memory_target     SCOPE=SPFILE;  -- must be 0
ALTER SYSTEM RESET memory_max_target SCOPE=SPFILE;
```

VERIFY AFTER STARTUP

```
-- alert log, 19c
Large Pages Information
Total Shared Global Region in Large Pages = 310 GB (100%)
Large Pages used by this instance: 158725 (310 GB)
Large Pages unused system wide = 45 (90 MB)
```

- *Anything under* ***100%*** *means you are on* `TRUE` *with a short pool*
- `HugePages_Free - HugePages_Rsvd` *in* `/proc/meminfo` *should be small; large = wasted RAM*
- *Why fail loud? A silent 4 KB fallback is the page-table trap from page 06*

SWIPE TO NEXT →

08 · SWAP

SWAP USED ≠ PAGING

SWAP USED > 0

Normal. Old idle pages parked on disk. Not a ticket.

VMSTAT SI / SO > 0, STEADILY

Paging in progress. A database paging its PGA is catastrophically slow.

```
vmstat 5 5      # si / so columns
sar -B 5 5      # pgscand/s, pgsteal/s = reclaim pressure
grep -E 'PageTables|HugePages_(Total|Free)|Swap' /proc/meminfo
```

SWAP SIZE DOC · INSTALLATION GUIDE FOR LINUX

RAM	SWAP
1 – 2 GB	1.5 × RAM
2 – 16 GB	= RAM
> 16 GB	**16 GB**

A 500 GB host gets **16 GB** of swap: a margin against the OOM killer, not a working area.

CAN REACH SWAP

PGA and process heaps · an SGA on 4 KB pages · AMM's `/dev/shm` granules

CAN NEVER REACH SWAP

An SGA in HugePages: the pages are pinned from boot. No `LOCK_SGA` needed.

Not mandated by Oracle: `vm.swappiness`. Setting it to 1–10 is common practice. PRACTICE

SWIPE TO NEXT →

09 · WHO MOVES THE MEMORY

AMM vs ASMM vs Manual

	Manual	ASMM (10G+)	AMM (11G+)
You set	Every `*_SIZE` pool	`SGA_TARGET`, `PGA_AGGREGATE_TARGET`, `PGA_AGGREGATE_LIMIT`	`MEMORY_TARGET`
Who moves it	Nobody	MMAN, between SGA pools	Oracle, between SGA **and** PGA
A `*_SIZE` you set	Is the size	Is a **floor**	Is a floor
Lives in	SysV shm	SysV shm	`/dev/shm` files, 4 KB pages
HugePages	Yes	**Yes**	**No** DOC
Use for	ASMM thrash cases	**All production**	Dev boxes under 4 GB

Production answer: ASMM + HugePages, with floors (`shared_pool_size`, `large_pool_size`) so MMAN cannot starve a pool during a spike. `MEMORY_TARGET` must be **0** — AMM and HugePages are mutually exclusive on Linux.

Two traps: a DBCA template that set `MEMORY_TARGET` leaves your pool idle and the SGA on `/dev/shm`; and AutoUpgrade carries `MEMORY_TARGET` through to 26ai. Make it a pre-check.

SWIPE TO NEXT →

10 · WORKED EXAMPLE

500 GB, RHEL, ASM, OLTP

STEP 1 · BUDGET THE HOST PRACTICE

ITEM	GB	BASIS
Physical RAM	500	
Kernel, slab, page cache (ASM)	− 20	~4% of RAM
Grid Infrastructure + ASM SGA	− 10	
Process overhead	− 20	2,000 × 10 MB
Agents, backup, sshd	− 10	
Headroom	− 25	5%
Available for SGA + PGA	**≈ 415**	

STEP 2 · SPLIT BY WORKLOAD PRACTICE

WORKLOAD	SGA : PGA	SGA	PGA TARGET	PGA LIMIT
OLTP	75 : 25	**310**	**105**	**105**
Mixed / consolidation	65 : 35	270	145	145
DW / batch / heavy PX	45 : 55	185	230	230

```
limit = MIN(2 × target, RAM − SGA − overhead)
OLTP: MIN(210, 500 − 310 − 85) = 105 G, not the 2× default
```

Then let `v$db_cache_advice` *and* `v$pga_target_advice` *correct the split after a representative week.*

SWIPE TO NEXT →

II · THE SETTINGS

What goes in the SPFILE and the OS

```
-- SGA: lives in HugePages
sga_max_size            = 310G
sga_target              = 310G
shared_pool_size        = 16G       -- floor
large_pool_size         = 4G        -- floor, RMAN / PX
-- PGA: anonymous 4 KB pages, swappable
pga_aggregate_target    = 105G
pga_aggregate_limit     = 105G      -- explicit, not 2x
memory_target           = 0         -- and memory_max_target
use_large_pages         = ONLY
processes               = 2000
```

OS setting	Value	Derivation
`vm.nr_hugepages`	**158,770**	310 × 512 + 50 slack
memlock (KB)	472,000,000	≈ 90% of RAM DOC
`kernel.shmmax`	332,859,965,440	310 GB in bytes
`kernel.shmall`	81,264,640	310 GB / 4096
swap · THP	16 GB · never	Installation Guide DOC

Consolidation node? Sum every instance's `SGA_MAX_SIZE` for the pool, and every instance's `PGA_AGGREGATE_LIMIT` for the budget. All instances `ONLY`.

SWIPE TO NEXT →

12 · SAME BOX, NO HUGEPAGES

SAME PARAMETERS. ONLY THE PAGE SIZE DIFFERS.

1. Startup fine. `free -g` fine. Everyone goes home.
2. ~1,500 sessions touch a 310 GB cache; each builds up to **620 MB** of page tables (`grep VmPTE /proc/<pid>/status`).
3. `PageTables` in `/proc/meminfo` reaches tens of GB and eats the 60 GB overhead budget.
4. The kernel reclaims anonymous memory: **PGA pages go to swap**. `vmstat` shows `so > 0`.
5. Processes stall on page faults; AWR shows `latch: cache buffers chains` and `free buffer waits`.
6. The SGA itself is swappable. `LOCK_SGA=TRUE` would pin it, but needs memlock anyway — at which point just use HugePages.
7. Every buffer-cache access misses the TLB: measurable CPU overhead, worse on NUMA.

WITHOUT
310 GB SGA · 4 KB pages
page tables: **up to ~620 MB / process**
SGA: swappable
TLB: 6 MB covered

WITH
310 GB SGA · 2 MB pages
page tables: **~1.2 MB / process**
SGA: pinned
TLB: 3 GB covered

This is why every large-SGA checklist starts with HugePages, and why `use_large_pages=ONLY` is a feature, not a risk.

SWIPE TO NEXT →

13 · TAKE IT WITH YOU

THREE THINGS TO CARRY AWAY

1. `SGA_MAX_SIZE` *drives the HugePages count.*
2. `PGA_AGGREGATE_LIMIT` *decides whether the host can swap.*
3. `MEMORY_TARGET` *must be* **0** *for either to work.*

```
nr_hugepages = CEIL(Σ SGA_MAX_SIZE / 2 MB) + 5 × instances
memlock (KB) ≥ nr_hugepages × 2048 (guide: ~90% of RAM)
shmall ≥ Σ SGA_MAX_SIZE / 4096
Σ SGA + Σ PGA_AGGREGATE_LIMIT + overhead ≤ RAM
```

I BUILT A CALCULATOR FOR THIS

ORACLE MEMORY SIZER · FREE, SINGLE HTML FILE

- *Enter RAM, instances, PROCESSES, workload, storage type, HugePages yes/no*
- *Get the SGA/PGA split, the PGA limit,* `vm.nr_hugepages`*, memlock, shmmax/shmall, swap*
- *Copy-ready SPFILE block and sysctl/limits block*
- *Every line tagged* DOC *or* PRACTICE*, and the ASMM-vs-AMM verdict with the reasons*

Open-source on GitHub — ***link in the first comment****.*

Size memory from evidence, not defaults.

`/proc/meminfo` · *the alert log's Large Pages block ·* `v$pgastat` · `v$sga_resize_ops` · *the advisories after a real week.*

SAVE + SHARE ✓